

Order, Evidence, and Effects: The Formal Semantics of Auths-Proof

bordumb · bordumbb@gmail.com

16 August 2026

ABSTRACT

Auths-Proof is a proof-carrying authorization system whose formal development spans algebra, denotational semantics, transition systems, bounded arithmetic, and refinement of shipping Rust. Its central safety idea is order-theoretic: delegation may preserve or reduce authority, but it may not enlarge the set of complete authorization facts admitted by a parent. This paper reconstructs that idea from the Lean 4 artifact under `formal/`, presents the model as mathematics rather than source-code commentary, and explains how the abstract relations connect to executable decisions.

The authority scope is a heterogeneous product of profile state, finite permission and audience sets, inclusive time intervals, an action-constraint preorder, optional budget ceilings, freshness policies, exact assurance identities, and pinned critical extensions. Lean proves component order laws, semantic monotonicity, canonical antisymmetry, downward closure of action coverage and evidence requirements, preservation of trust roots and critical extensions, strict decrease of delegation depth, finiteness of chains, unique accepted transitions, and sound and complete decision procedures. A separate three-valued algebra treats denial, uncertainty, and authorization as an ordered chain and proves threshold-classification properties. Product and lifecycle modules formalize configuration equality, checked arithmetic, capacity conservation, replay classification, credential ordering, provider-call entry, and recovery from unknown outcomes.

To connect specification with implementation, Charon and Aeneas translate selected pure safe Rust functions into Lean. Representation maps and weakest-precondition proofs relate translated strings, bounded vectors, sets, intervals, constraints, budgets, and state machines to the handwritten model. The current assurance inventory contains 121 compiled declarations: 66 rich-authority claims, 4 production-refinement claims, 13 product claims, 22 lifecycle claims, and 16 composition claims. The artifact also records its limits: canonical decoding, cryptography, external evidence, stores, and provider behavior are not proved; representation validity is a premise; and the checked-in translation has an explicitly proved stale optional-budget case. The result is not whole-program verification. It is a precise, inspectable refinement argument for the security-critical pure semantics.

Contents

1 Introduction	3	10 Refinement from shipping Rust to mathematical semantics	18
1.1 Contributions of the formal artifact	3	10.1 Three semantic levels	18
1.2 Reading the claims correctly	4	10.2 Aeneas weakest-precondition judgments	19
1.3 The proof architecture	4	10.3 Representation maps	19
2 Artifact map and methodological boundary	4	10.4 Authority refinement theorems	19
2.1 What the formal core intentionally excludes	5	10.5 Lifecycle and bounded-policy refinement	20
2.2 Snapshot of the audited theorem inventory	5	10.6 The explicit stale-translation theorem	20
3 The three-valued decision algebra	6	11 Assurance engineering around the proofs	20
3.1 Conjunctive and disjunctive composition	6	11.1 Public theorem inventory	21
3.2 Threshold classification	7	11.2 Qualified mechanical translation	21
3.3 Outcomes and diagnostics	7	11.3 Source closure	21
4 Authority as a heterogeneous product order	7	11.4 Generated vectors and mutation witnesses	21
4.1 Finite sets and validity intervals	8	11.5 Why proof and model checking coexist	22
4.2 Profile selection	9	12 Trusted computing base and residual assumptions	22
4.3 Action constraints	9	12.1 Logical foundation	22
4.4 Optional budgets	9	12.2 Translation foundation	22
4.5 Status, assurance, and critical extensions	10	12.3 Representation foundation	23
5 Denotational authorization semantics	10	12.4 Environmental foundation	23
5.1 Actions and evidence	10	12.5 Stateful implementation foundation	23
5.2 Structural soundness	11	12.6 Claim matrix	23
5.3 Preorder, canonical partial order, and equivalence	11	13 Cross-disciplinary interpretation	24
5.4 Decidability	11	13.1 Order theory: authorization as a monotone semantics	24
6 Delegation as a well-founded transition system	12	13.2 Denotational semantics: meaning before implementation	24
6.1 Rootedness and linkage	12	13.3 Type theory: invalid states as missing constructors	24
6.2 Accepted transition	12	13.4 Program logic: executable refinements	24
6.3 Finiteness and transitive attenuation	12	13.5 Distributed systems: uncertainty is state	24
6.4 Terminal coverage and lineage	13	13.6 Security engineering: negative-space theorems	24
6.5 Critical extensions and falsifiability	13	14 Limitations and open proof obligations	25
7 Executable decisions and ordered diagnostics	14	14.1 Regenerate the stale authority translation	25
7.1 Grant evaluation	14	14.2 Close constructor-to-view refinement	25
7.2 Author-planning evaluation	14	14.3 Strengthen plan semantics	25
7.3 Terminal coverage evaluation	14	14.4 Connect state-machine relations to stores	25
8 Product policy: commitments, eligibility, and arithmetic	15	14.5 Separate information-flow properties	25
8.1 Semantic and configuration commitments	15	14.6 Expand domain-specific evaluator proofs vertically	25
8.2 Three-way eligibility	15	15 Related work	25
8.3 Closed evaluators and fixed-context tightening	16	16 Conclusion	26
8.4 Unit-aware checked arithmetic	16	Appendix A. Complete scope relation	27
9 Effect lifecycle as a guarded transition system	16	Appendix B. Delegation proof skeleton	27
9.1 States, operations, and gates	16	Appendix C. Lifecycle transition summary	28
9.2 Safety ordering	17	Appendix D. Public theorem families	28
9.3 Capacity algebra	17	Appendix E. Module dependency guide	29
9.4 Replay classification	18	References	29

1 Introduction

Authorization is often presented as a Boolean predicate over a subject, an operation, and a resource. That abstraction is useful, but it is too coarse for a system in which authority is delegated, narrowed, bound to exact action bytes, evaluated under uncertain evidence, and ultimately converted into an irreversible external effect.

Auths-Proof separates four questions:

1. **Authenticity:** who produced a signed statement?
2. **Authority:** which actions can be justified from a trust root?
3. **Eligibility:** does a domain-specific policy admit this action in this context?
4. **Effect:** was the authorized action reserved, attempted, observed, and settled without duplication?

The formal directory concentrates on the pure semantics that make these questions composable. It does not model public-key cryptography, byte decoding, networks, clocks, databases, or providers. Instead, it gives mathematically precise meanings to attenuation, coverage, evidence, decision composition, bounded product policy, and effect lifecycle. It then connects selected shipping Rust predicates to those meanings by mechanical translation.

The organizing judgment of the paper is:

If a child authority is structurally below its parent, every complete authorization fact admitted by the child is admitted by the parent.

Writing $S_c \leq S_p$ for structural attenuation and $\llbracket S \rrbracket$ for the set of complete authorization facts admitted by a scope, the fundamental implication is

$$S_c \leq S_p \implies \llbracket S_c \rrbracket \subseteq \llbracket S_p \rrbracket. \quad (1)$$

Equation (1) is the semantic form of least authority. A delegate can keep the same authority or move downward, but cannot create a fact that becomes authorized only after delegation. This connects order theory to access control: the order is not chosen for elegance alone; it is justified by set containment of admitted behaviors.

The work belongs to several research traditions. Trust-management systems make delegation and authority explicit (Abadi et al. 1993; Blaze, Feigenbaum, and Lacy 1996). Capability systems and caveat-based tokens motivate monotone restriction (Birgisson et al. 2014). Proof-carrying systems motivate portable evidence checked by a small verifier (Necula 1997; Appel and Felten 1999). Order theory supplies preorders, partial orders, products, and monotone maps (Davey and Priestley 2002). Program logic supplies preconditions, postconditions, and refinement (Hoare 1969; Dijkstra 1975). Lean 4 provides the machine-checked foundation (Moura and Ullrich 2021), while Aeneas provides a route from safe Rust to functional Lean (Ho and Protzenko 2022).

1.1 Contributions of the formal artifact

The development makes six concrete contributions.

A heterogeneous authority order. Authority is not reduced to a scalar. Finite sets use inclusion, intervals use containment, action constraints use a constructor-sensitive preorder, optional budgets have an unbounded top, freshness requirements order by method and age, assurance is invariant, and critical extensions become exactly pinned after their first declaration.

A denotational safety theorem. Structural attenuation implies semantic attenuation: admission is downward closed for both actions and evidence.

A well-founded delegation semantics. Accepted edges preserve one trust root, preserve pinned critical extensions, link exact grant identifiers, strictly reduce remaining depth, and determine a unique child state.

A three-valued composition algebra. Denial, indeterminacy, and authorization form a finite chain. Conjunction, disjunction, and threshold classification preserve uncertainty rather than collapsing missing evidence into permission.

A formal lifecycle for effects. The model separates decision, reservation, intent, credential authorization, attempt, provider entry, settlement, unknown outcome, and reconciliation. It proves key ordering and capacity invariants.

A production refinement chain. Pure shipping Rust is translated to Lean, then proved against the readable specification under explicit representation premises. The proof inventory, statement hashes, axiom dependencies, source closure, generated vectors, and mutation operators are themselves checked artifacts.

1.2 Reading the claims correctly

The paper uses three labels.

- **Definition** means a mathematical reconstruction of a Lean definition.
- **Checked theorem** means the named proposition is compiled in the current theorem inventory.
- **Interpretation** means a consequence or explanatory view useful to a human reader; it should not be confused with a separately inventoried Lean theorem.

This distinction matters. For example, the three truth values form a finite chain and the implementations of `all` and `any` coincide with minimum and maximum on that chain. Lean directly inventories commutativity, associativity, and idempotence; this paper may use lattice language to explain those facts, but it does not pretend that every familiar lattice law has its own exported theorem.

1.3 The proof architecture

The artifact is not one monolithic proof. It is a graph of specifications, implementations, representation maps, and audit evidence.

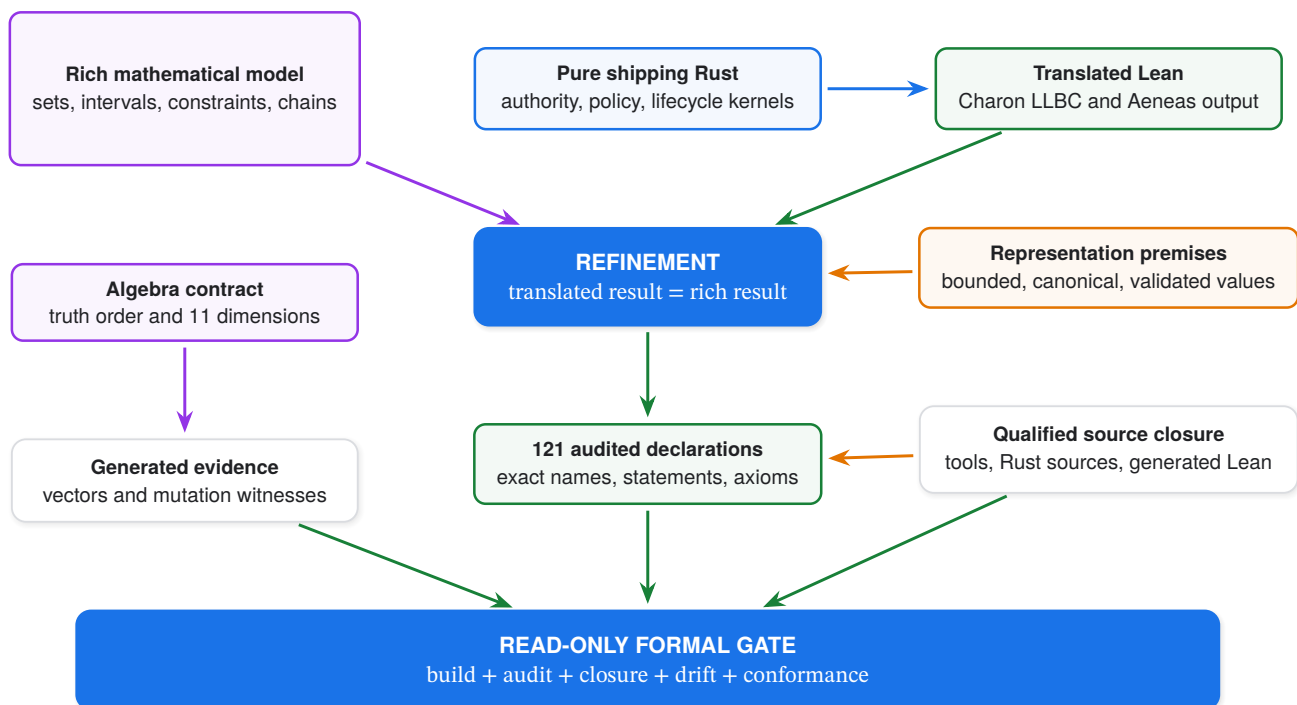


Figure 1: The evidence graph. The readable mathematical model and the mechanically translated production functions meet in refinement theorems. The assurance layer audits both the propositions and the source closure on which they depend.

2 Artifact map and methodological boundary

The Lean project is organized by mathematical concern rather than by one end-to-end verifier function.

Layer	Principal modules	Mathematical role
Finite algebra	Generated/Algebra, Base, Composition	Truth chain, conjunction, disjunction, thresholds, plan measures
Rich authority	Rich/Types, Rich/Semantics, Rich/Theorems	Scope order, denotation, delegation, coverage, decisions
Focused authority claims	Authority, Attenuation	Trust-root and critical-extension consequences

Layer	Principal modules	Mathematical role
Product policy	Product/*	Commitments, checked arithmetic, eligibility, tightening
Effect lifecycle	Lifecycle/*	State transitions, capacity, replay, reconciliation
Rust refinement	Refinement/Production	Representation maps and equivalence to translated authority Rust
Assurance	Theorems, AssuranceAudit, manifest	Public inventory, exact statements, axiom audit, source closure
Qualification	qualification/aeneas/*	Pinned translation outputs, external models, executable cases

The top-level `Auths.lean` imports the model. `Auths.Theorems` enumerates the public theorem declarations. `Auths.AssuranceAudit` queries Lean’s compiled environment, prints each declaration’s actual type, and computes its transitive axiom set. This avoids a common anti-pattern: treating the presence of a theorem-like string in source code as evidence that the intended theorem still exists.

2.1 What the formal core intentionally excludes

The model receives already validated semantic values and explicit facts. It does not prove:

- canonical CBOR decoding or byte-for-byte re-encoding;
- signature schemes, key custody, or trust-registry behavior;
- evidence acquisition, wall-clock truth, or network freshness;
- database atomicity, crash durability, or concurrency control;
- external provider determinism or the truth of provider observations;
- the Rust compiler’s machine-code output; or
- complete verifier control flow.

These exclusions are not incidental. They preserve a pure kernel in which definitions are total and explicit. The complete system claim is therefore a composition of proof, translation, generated conformance, testing, and trusted boundaries rather than a single universal theorem.

Claim boundary. Lean proves the pure relations and selected translated Rust functions under their premises. It does not prove that arbitrary bytes are valid `Auths` objects, that external facts are true, or that a remote effect occurred. Those propositions require distinct evidence.

2.2 Snapshot of the audited theorem inventory

The current `theoremInventory` contains 121 declarations.

Family	Count	Examples
Rich authority	66	component laws, semantic containment, chains, diagnostics
Production refinement	4	author scope, terminal coverage, delegation, explicit budget gap
Product policy	13	commitments, eligibility, arithmetic, translated arithmetic
Lifecycle	22	capacity, replay, effect ordering, translated transition kernel
Composition	16	truth algebra, thresholds, plan measures

Auxiliary definitions and lemmas may compile without appearing in this public inventory. The manifest is therefore a

claim surface, not a count of every fact Lean knows.

3 The three-valued decision algebra

Authorization under incomplete evidence is not naturally Boolean. A negative result may be permanent, or it may mean that a trustworthy fact is not yet available. Auths-Proof uses

$$\mathbb{T}_3 = \{D, U, A\}, \quad D \sqsubset U \sqsubset A. \quad (2)$$

The Lean function `Truth.rank` embeds this chain into $\mathbb{N}$:

$$\rho(D) = 0, \quad \rho(U) = 1, \quad \rho(A) = 2,$$

and defines $x \sqsubseteq y$ by $\rho(x) \leq \rho(y)$. Injectivity of ρ is proved, so rank equality recovers truth equality.

3.1 Conjunctive and disjunctive composition

Define

$$x \wedge_3 y = \min_{\sqsubseteq}(x, y), \quad x \vee_3 y = \max_{\sqsubseteq}(x, y). \quad (3)$$

Operationally, `all` denies if either branch denies, otherwise remains indeterminate if either branch is indeterminate, and authorizes only when both authorize. `any` authorizes if either branch authorizes, otherwise remains indeterminate if either is indeterminate, and denies only when both deny.

x	y	$x \wedge_3 y$	$x \vee_3 y$
D	D	D	D
D	U	D	U
D	A	D	A
U	U	U	U
U	A	U	A
A	A	A	A

Checked theorem family: binary composition

Lean proves commutativity, associativity, and idempotence for both `all` and `any`. It also proves that the two-branch threshold at one equals `any`, the threshold at two equals `all`, and increasing the threshold from one to two cannot raise the result in the truth order.

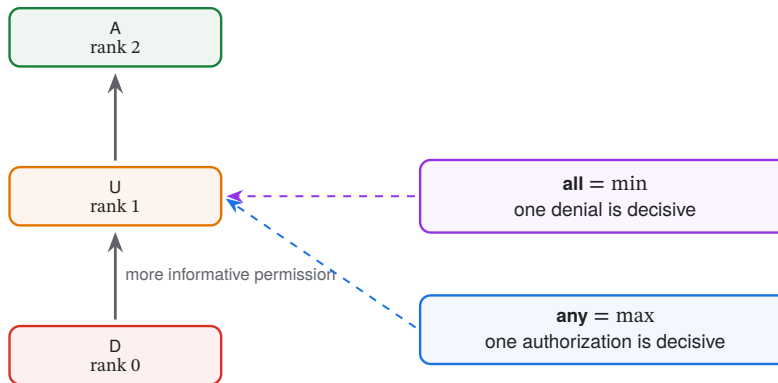


Figure 2: The truth chain. Indeterminacy is neither authorization nor permanent denial. Conjunction takes the lower result; disjunction takes the higher result.

3.2 Threshold classification

Let k be the number of authorizing branches required, a the number already authorized, and u the number indeterminate. The generated classifier is

$$\Theta(k, a, u) = \begin{cases} A, & a \geq k, \\ U, & a < k \leq a + u, \\ D, & a + u < k. \end{cases} \quad (4)$$

These regions are exhaustive and disjoint. The first says the threshold has already been met. The second says authorization is not yet established but remains reachable if enough unknown branches resolve positively. The third says even the optimistic completion cannot meet the threshold.

Checked theorem family: threshold soundness

authorized_implies_threshold_met proves $\Theta = A \Rightarrow k \leq a$. denied_implies_threshold_impossible proves $\Theta = D \Rightarrow a + u < k$. indeterminate_implies_threshold_reachable proves $\Theta = U \Rightarrow a < k \wedge k \leq a + u$.

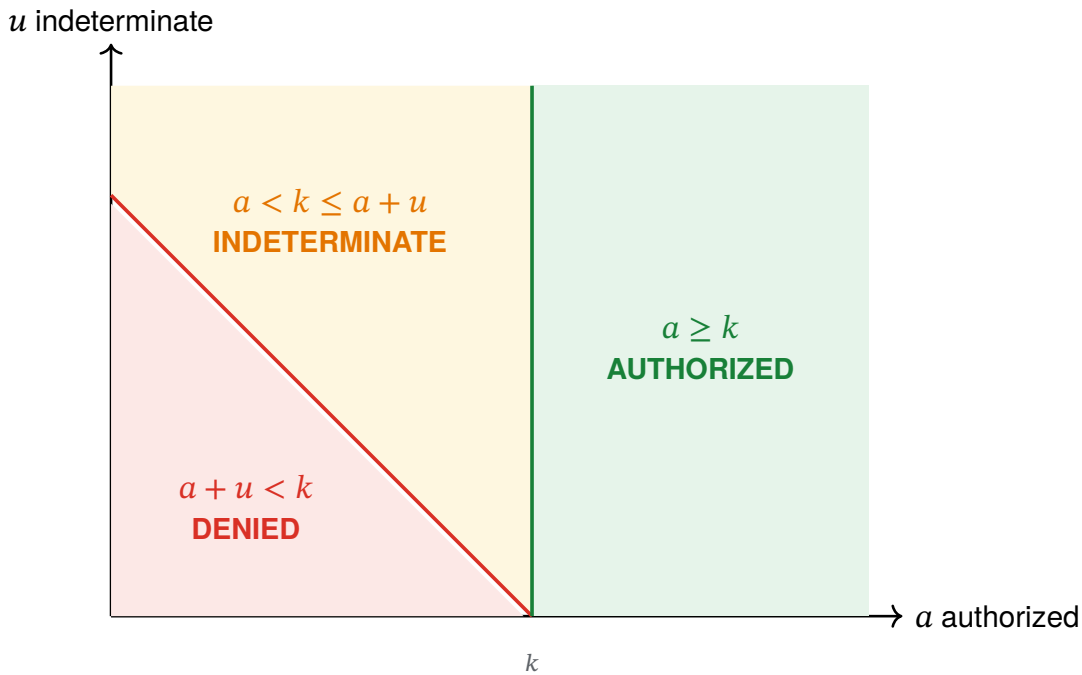


Figure 3: Threshold regions for fixed k . The classifier separates proved success, reachable success, and impossible success without assuming how unknown branches will resolve.

3.3 Outcomes and diagnostics

Outcome enriches truth with stable numerical diagnostics:

$$\text{Outcome} = \text{Authorized} \mid \text{Denied}(c) \mid \text{Indeterminate}(c) \mid \text{StructurallyInvalid}(c).$$

Projection to $\mathbb{T}_3$ maps structural invalidity to denial. The helper `canonicalCode(c_1, c_2) = \min(c_1, c_2)` is commutative, which gives a canonical binary diagnostic independent of operand order. This is a deliberately narrow result: the formal Plan module defines leaf enumeration and node count, but does not yet export a general recursive authorization-plan correctness theorem. Its inventory claims about `visit` and `cost` are definitional equalities.

4 Authority as a heterogeneous product order

The rich model is parameterized by a vocabulary V of opaque carrier types:

$V = (\text{Principal}, \text{Profile}, \text{Permission}, \text{Audience}, \text{Digest}, \text{BudgetAlg}, \text{StatusMethod}, \text{Assurance}, \text{GrantId}, \text{ExtId}, \text{ExtBody}).$

Each carrier has decidable equality and no other assumed structure. In particular, the mathematical model does not order principals, permissions, or digests by an arbitrary numerical encoding. Production refinement later instantiates these carriers with exact byte-oriented representations.

An authority scope is the tuple

$$S = (\Phi, P, I, A, C, B, T, Q, X), \quad (5)$$

where

- Φ is profile-selection state;
- P is a finite permission set;
- I is a well-formed inclusive validity interval;
- A is a finite audience set;
- C is an action-body constraint;
- B is an optional budget ceiling;
- T is a status-evidence policy;
- Q is an assurance-policy identity; and
- X is an optional pinned critical-extension sequence.

The attenuation relation $S_c \leq S_p$ is the conjunction of a relation for every coordinate. The coordinates are intentionally heterogeneous.

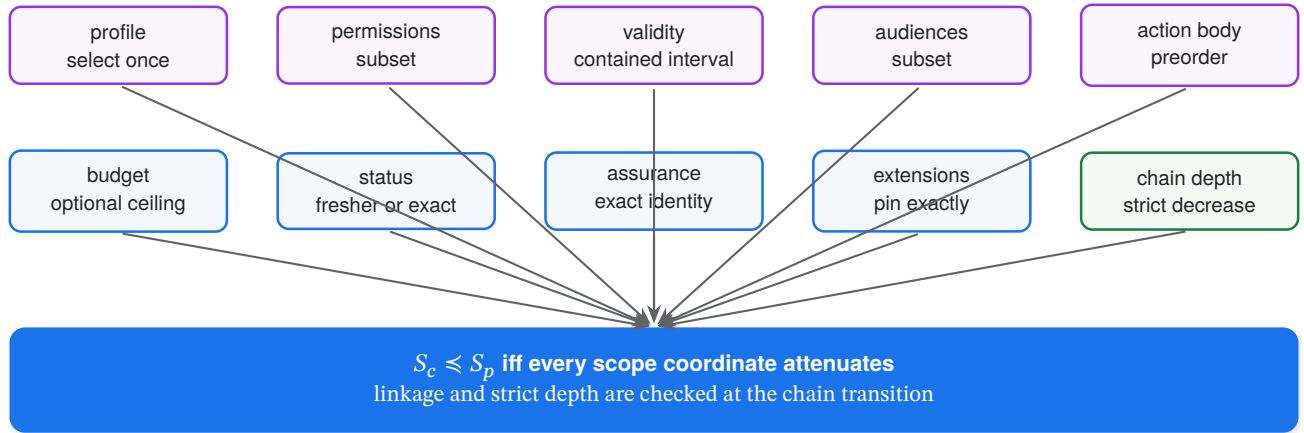


Figure 4: The authority product. The scope order is a conjunction of relations chosen for each domain. Strict depth is not a partial-order coordinate; it is a well-founded transition guard.

4.1 Finite sets and validity intervals

Permissions and audiences use finite-set inclusion:

$$P_c \leq P_p \iff P_c \subseteq P_p, \quad A_c \leq A_p \iff A_c \subseteq A_p. \quad (6)$$

An inclusive window is $I = [s, f]$ with a constructor proof $s \leq f$. Containment is

$$[s_c, f_c] \leq [s_p, f_p] \iff s_p \leq s_c \wedge f_c \leq f_p. \quad (7)$$

Lean proves reflexivity, transitivity, and antisymmetry for both finite-set inclusion and interval containment. It also proves membership and interval coverage monotone: an item or action window covered by a child remains covered by a parent.

4.2 Profile selection

A profile scope is (R, s) , where R is the invariant root-allowed set and $s \in \{\text{none}\} \cup R$ is an optional selected profile. The type stores the proof that every selected profile belongs to R . Its order preserves R exactly and permits only

$$\text{none} \rightarrow \text{none}, \quad \text{none} \rightarrow \text{some}(p), \quad \text{some}(p) \rightarrow \text{some}(p). \quad (8)$$

Selection is therefore a one-way refinement. It cannot be cleared or changed after it is fixed. Before selection, any $p \in R$ is covered; after selection, only the selected profile is covered.

4.3 Action constraints

The action-body language is closed:

$$C ::= \text{Any} \mid \text{Exact}(d) \mid \text{Allowed}(D),$$

where D is a finite digest set. Its denotation is

$$\llbracket \text{Any} \rrbracket = \mathcal{D}, \quad \llbracket \text{Exact}(d) \rrbracket = \{d\}, \quad \llbracket \text{Allowed}(D) \rrbracket = D. \quad (9)$$

The structural relation follows denotational inclusion but respects the constructors explicitly:

$$\begin{aligned} C &\leq \text{Any} \quad \text{always,} \\ \text{Exact}(x) &\leq \text{Exact}(y) \iff x = y, \\ \text{Exact}(x) &\leq \text{Allowed}(Y) \iff x \in Y, \\ \text{Allowed}(X) &\leq \text{Exact}(y) \iff X \subseteq \{y\}, \\ \text{Allowed}(X) &\leq \text{Allowed}(Y) \iff X \subseteq Y. \end{aligned} \quad (10)$$

This relation is reflexive and transitive. It is not structurally antisymmetric on raw syntax because $\text{Exact}(d)$ and $\text{Allowed}(\{d\})$ admit the same digest and attenuate each other. The model therefore proves two different statements:

1. mutual attenuation implies extensional equivalence of allowed digests; and
2. if singleton allow-lists are excluded by a canonicity predicate, mutual attenuation implies constructor equality.

This is an important formal-methods lesson: quotient-like semantic equality and representation equality should not be conflated.

4.4 Optional budgets

A budget ceiling is a pair (α, n) of an opaque algebra identity and a natural-number value. The optional order treats absence as unbounded top:

$$B_c \leq B_p \iff \begin{cases} \text{true,} & B_p = \text{none}, \\ \text{false,} & B_c = \text{none}, B_p = \text{some}(\alpha, n), \\ \alpha_c = \alpha_p \wedge n_c \leq n_p, & B_c = \text{some}(\alpha_c, n_c), B_p = \text{some}(\alpha_p, n_p). \end{cases} \quad (11)$$

Terminal coverage has a different argument orientation. A ceiling covers a request when

$$\text{budgetCovers}(B, R) \iff \begin{cases} \text{true,} & B = \text{none}, \\ \text{false,} & B = \text{some}(-), R = \text{none}, \\ \alpha_R = \alpha_B \wedge n_R \leq n_B, & B = \text{some}(\alpha_B, n_B), R = \text{some}(\alpha_R, n_R). \end{cases} \quad (12)$$

The second case is security-critical. An absent request states no action-level bound, so it is not covered by a present ceiling. Lean proves budget-order reflexivity, transitivity, antisymmetry, and monotonicity of coverage.

4.5 Status, assurance, and critical extensions

A status policy is either

$$\text{ExpiryOnly} \quad \text{or} \quad \text{SnapshotRequired}(m, \delta),$$

where m is an opaque method and $\delta > 0$ is a maximum age. Every policy is below `ExpiryOnly`. Two snapshot policies compare only when their methods are equal, and the child maximum age is no greater than the parent's. Thus a fresher requirement is narrower. Lean proves that satisfaction is monotone from child to parent.

Assurance is not modeled as a lattice. It is preserved by exact identity: $Q_c = Q_p$. This deliberately avoids inventing an ordering between assurance schemes.

Critical extensions follow a lock-in relation. Let X be either `none` or a canonical ordered sequence of at most 32 identifier-payload pairs. Then

$$X_c \leq X_p \iff \begin{cases} \text{true,} & X_p = \text{none}, \\ \text{false,} & X_c = \text{none}, X_p = \text{some}(E), \\ E_c = E_p, & X_c = \text{some}(E_c), X_p = \text{some}(E_p). \end{cases} \quad (13)$$

The first accepted edge may declare a critical-extension sequence. Every later edge must preserve it exactly. Unlike permissions and audiences, extensions are represented by an ordered list because the shipping predicate compares canonical vectors positionally. Distinct identifiers and the cardinality bound are stored as constructor obligations.

5 Denotational authorization semantics

The product order becomes a security statement only after scopes are given a meaning. Auths-Proof defines that meaning over actions and trusted evidence facts.

5.1 Actions and evidence

An action carries

$$a = (\text{actor}, \text{terminalGrant}, \text{profile}, \text{permission}, \text{window}, \text{audience}, \text{bodyDigest}, \text{requestedBudget}).$$

Evidence facts carry an optional status method, a status age, and an assurance identity. Complete authorization facts are a pair $F = (a, e)$.

Scope-level action coverage is

$$\begin{aligned} \text{ActionCovers}(S, a) \iff & \text{profileAllows}(\Phi, a.\text{profile}) \\ & \wedge a.\text{permission} \in P \\ & \wedge a.\text{window} \subseteq I \\ & \wedge a.\text{audience} \in A \\ & \wedge \text{constraintAllows}(C, a.\text{bodyDigest}) \\ & \wedge \text{budgetCovers}(B, a.\text{requestedBudget}). \end{aligned} \quad (14)$$

Evidence satisfaction is

$$\text{EvidenceSatisfied}(S, e) \iff \text{statusSatisfied}(T, e) \wedge e.\text{assurance} = Q. \quad (15)$$

Finally,

$$\text{Admits}(S, F) \iff \text{ActionCovers}(S, F.\text{action}) \wedge \text{EvidenceSatisfied}(S, F.\text{evidence}). \quad (16)$$

Define the denotation

$$\llbracket S \rrbracket = \{F \mid \text{Admits}(S, F)\}. \quad (17)$$

This is the bridge between access control and order theory.

5.2 Structural soundness

The semantic attenuation relation is extensional:

$$S_c \leq_{\text{sem}} S_p \iff \forall F. \text{Admits}(S_c, F) \Rightarrow \text{Admits}(S_p, F). \quad (18)$$

Equivalently, $\llbracket S_c \rrbracket \subseteq \llbracket S_p \rrbracket$.

Theorem 5.1 (Structural attenuation is semantically sound). *If $S_c \leq S_p$, then $S_c \leq_{\text{sem}} S_p$.*

The Lean proof factors by concern. `action_coverage_downward_closed` uses profile monotonicity, set membership monotonicity, interval transitivity, constraint allowance monotonicity, and budget coverage monotonicity. `evidence_requirements_downward_closed` uses status-satisfaction monotonicity and assurance equality. `complete_admission_downward_closed` combines them. `structural_scope_le_implies_semantic_attenuation` packages the result as Equation (18).

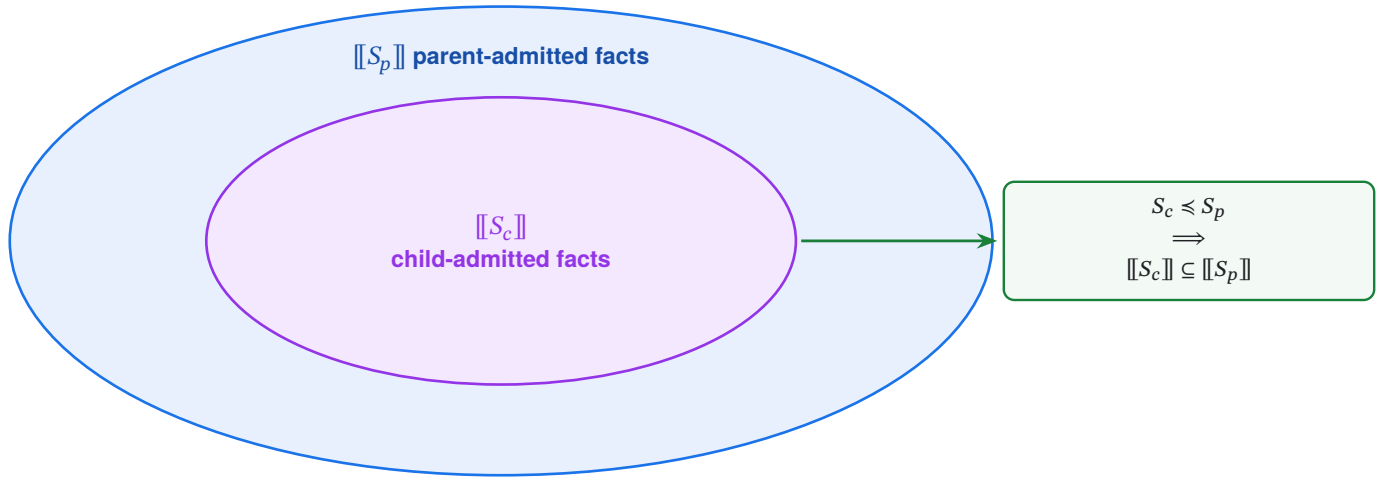


Figure 5: Denotational attenuation. A narrower scope may remove authorized facts, but cannot add a fact outside the parent’s authorization denotation.

5.3 Preorder, canonical partial order, and equivalence

Semantic attenuation is reflexive and transitive because subset is reflexive and transitive. Mutual semantic attenuation defines extensional equivalence. The structural scope relation is also reflexive and transitive.

Raw action-constraint syntax prevents global structural antisymmetry, as noted in Section 4.3. Under canonical action constraints, however, every component is antisymmetric. Lean then proves

$$S_1 \leq S_2 \wedge S_2 \leq S_1 \implies S_1 = S_2. \quad (19)$$

Thus the development uses the correct algebraic level for each question:

- a preorder for raw representations;
- semantic equivalence for denotations; and
- a partial order for canonical representatives.

5.4 Decidability

Every carrier supplies decidable equality, and every component relation is decidable. `structuralScopeLeDecide` computes a Boolean, and Lean proves

$$\text{structuralScopeLeDecide}(S_c, S_p) = \text{true} \iff S_c \leq S_p. \quad (20)$$

This theorem is small but architecturally important. The mathematical relation is not merely axiomatized; it has a decision procedure whose positive result is exactly the declared V1 relation.

6 Delegation as a well-founded transition system

Scope order alone does not express identity linkage, trust roots, grant lineage, or chain termination. These live in `ChainState`:

$$q = (r, s, S, d, g), \quad (21)$$

where r is the root principal, s the current subject, S the authority scope, $d \in \mathbb{N}$ the remaining depth, and g an optional identifier of the last applied grant.

6.1 Rootedness and linkage

The local predicate

$$\text{rooted}(q) \iff g \neq \text{none} \vee r = s \quad (22)$$

states that either an accepted edge has already been applied or a fresh state still speaks for its root. A proposed grant h preserves the root when

$$\text{rootPreserved}(q, h) \iff \text{rooted}(q) \wedge h.\text{issuer} = s. \quad (23)$$

It is linked when, additionally, $h.\text{parent} = g$. These are local semantic facts; the formal module does not reverify signatures or reconstruct a proof graph.

The first edge of a fresh chain must be issued by the root itself. Every accepted edge is issued by the current subject and copies the root into its child. An unrooted state delegates nothing and authorizes no terminal action.

6.2 Accepted transition

A grant carries its issuer, new subject, selected profile, narrowed scope coordinates, remaining depth, parent grant identifier, and critical extensions. It passes scope and depth checks when

$$d > 0 \wedge h.\text{depth} < d \wedge \text{GrantScopeChecks}(S, h). \quad (24)$$

If linked and checked, `acceptedNextState` deterministically constructs

$$q' = (r, h.\text{subject}, \text{acceptedScope}(S, h), h.\text{depth}, \text{some}(h.\text{id})). \quad (25)$$

The proposition `delegates(q, id, h, q')` says exactly that linkage holds and q' is this constructed state for some proof of the checks.

Checked theorem family: accepted delegation

Lean proves that accepted evaluation is equivalent to `delegates`, that the accepted state is unique, that the child scope is below the parent scope, that subject and last-grant fields update exactly, and that remaining depth strictly decreases.

6.3 Finiteness and transitive attenuation

The strict relation

$$q' <_d q \iff q'.\text{depth} < q.\text{depth}$$

is well-founded by the natural-number measure. For a delegation chain starting at q_0 with successor list L , Lean proves

$$|L| \leq q_0.\text{depth}. \quad (26)$$

Hence no accepted chain can be infinite, even if every scope coordinate stays unchanged. Strict depth is a transition measure, not another reflexive authority coordinate.

For two accepted edges $q_0 \rightarrow q_1 \rightarrow q_2$, transitivity of the product order gives

$$q_2.\text{scope} \leq q_0.\text{scope}. \quad (27)$$

More generally, every reachable state remains under the starting root, is rooted, and preserves any critical-extension sequence pinned at the start.

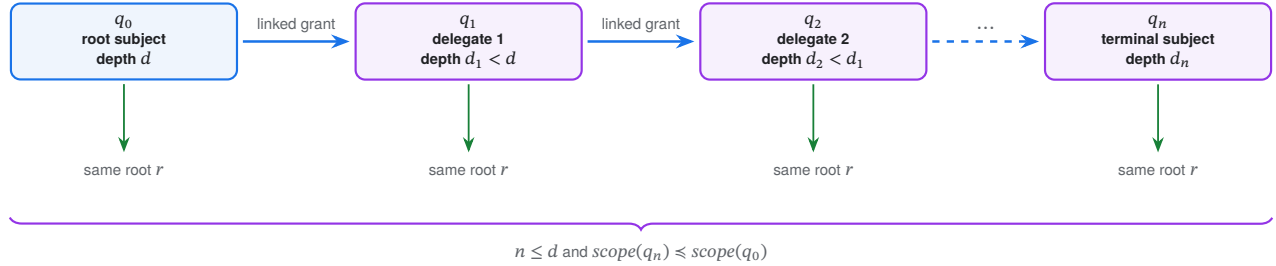


Figure 6: A delegation chain. Root identity is invariant, scope moves downward, and a natural-number variant makes the chain finite.

6.4 Terminal coverage and lineage

Terminal coverage strengthens scope-level action coverage with chain facts:

$$\begin{aligned} \text{TerminalCovers}(q, a) \iff & \text{rooted}(q) \\ & \wedge a.\text{actor} = q.\text{subject} \\ & \wedge a.\text{terminalGrant} = q.\text{lastGrant} \\ & \wedge \text{ActionCovers}(q.\text{scope}, a). \end{aligned} \quad (28)$$

The evaluator checks these predicates in a fixed first-failure order. Lean proves

$$\text{evaluateCoverage}(q, a) = \text{authorized} \iff \text{TerminalCovers}(q, a). \quad (29)$$

It also proves every returned denial is incompatible with terminal coverage. If a child action is covered after an accepted edge, scope monotonicity implies that the action was also within the parent's scope. The actor and terminal grant naturally differ across the edge, so the theorem is stated for `actionCovers`, not for full terminal linkage.

6.5 Critical extensions and falsifiability

The generated attenuation projection contains eleven Booleans:

$$(b_{\text{root}}, b_{\text{depth}}, b_{\text{profile}}, b_{\text{perm}}, b_{\text{valid}}, b_{\text{aud}}, b_{\text{action}}, b_{\text{budget}}, b_{\text{status}}, b_{\text{assurance}}, b_{\text{ext}}).$$

Acceptance is their conjunction. A Boolean dimension would be vacuous if the projection filled it with the literal `true`. The formal development therefore proves not only that accepted projection implies root and extension requirements, but that each dimension is exact and falsifiable.

For root preservation,

$$b_{\text{root}} = \text{true} \iff \text{rootPreserved}(q, h), \quad (30)$$

and a foreign issuer forces $b_{\text{root}} = \text{false}$. For critical extensions,

$$b_{\text{ext}} = \text{true} \iff \text{extensionsLe}(\text{some}(h.\text{extensions}), q.\text{scope}.\text{extensions}), \quad (31)$$

and an altered pinned sequence forces $b_{\text{ext}} = \text{false}$. No other true dimension can rescue either failure because the aggregator is conjunction.

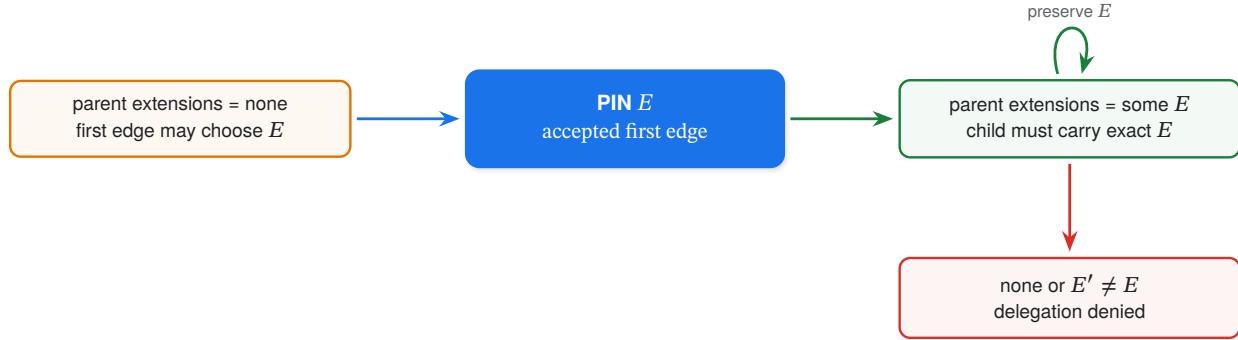


Figure 7: Critical-extension lock-in. The anchor is initially uncommitted. Once an accepted edge selects a sequence, later edges may neither remove nor alter it.

7 Executable decisions and ordered diagnostics

The rich relations are propositions, but the system must return a stable decision. The formal development defines three evaluators with explicit first-failure order.

7.1 Grant evaluation

`evaluateGrant(q, id, h)` returns either an accepted next state or one of two diagnostics:

`BrokenGrantChain` or `DelegationExpanded`.

It tests linkage first. If linkage fails, the chain diagnostic is returned. If linkage succeeds but scope or depth checks fail, expansion is returned. If both hold, it constructs the unique accepted state.

Lean proves both extensional correctness and diagnostic partition:

$$\exists q'. \text{evaluateGrant}(q, id, h) = \text{accepted}(q') \iff \text{linked}(q, h) \wedge \text{scopeDepthChecks}(q, h), \quad (32)$$

$$\begin{aligned} \text{BrokenGrantChain} &\iff \neg \text{linked}(q, h), \\ \text{DelegationExpanded} &\iff \text{linked}(q, h) \wedge \neg \text{scopeDepthChecks}(q, h). \end{aligned} \quad (33)$$

7.2 Author-planning evaluation

Before signing or invoking custody, `evaluateAuthorScope` compares a proposed child scope to a parent. It checks dimensions in this order:

`profile` `precpermissions` `precvalidity` `precaudiences` `precactionConstraint` `precbudget` `precdepth` `precstatus` `precassurance` `precextensions`. (34)

The theorem `author_planning_diagnostic_sound_complete` proves that acceptance is equivalent to structural scope attenuation plus strict valid depth:

$$\text{evaluateAuthorScope} = \text{accepted} \iff S_c \leq S_p \wedge 0 < d_p \wedge d_c < d_p. \quad (35)$$

The fixed order is functional determinism, not a timing-security claim. A system may expose the first diagnostic for explainability while separately considering whether timing and cache behavior reveal sensitive policy facts.

7.3 Terminal coverage evaluation

`evaluateCoverage` checks root and lineage facts as one leading guard, then permission membership, validity containment, audience membership, action-body constraint, and budget coverage. The leading guard maps actor mismatch, terminal-grant mismatch, invalid profile selection, and unrooted state to `BrokenGrantChain`; later dimensions receive more specific diagnostics.

This design makes the success theorem strong and the diagnostic vocabulary stable. It does not claim that every semantic falsehood has a unique reason; when several checks fail, the evaluator intentionally selects the earliest.

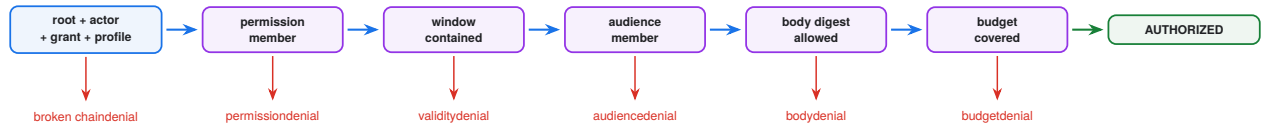


Figure 8: Ordered terminal-coverage diagnostics. Success means every predicate holds. Failure reports the earliest false dimension in the specified order.

8 Product policy: commitments, eligibility, and arithmetic

The authority kernel answers whether delegated authority covers an action. Product policy answers whether a closed, versioned domain evaluator admits it under a fixed context. The formal modules keep that distinction explicit.

8.1 Semantic and configuration commitments

A semantic identity is a nonempty byte sequence. A digest is exactly 32 bytes. A policy commitment records

$$(type, version, canonicalization, policyDigest, evaluatorSemantics),$$

with a proof that the version is positive. A configuration commitment records

$$(semantics, canonicalization, digest, implementation?), \quad (36)$$

where the implementation identity may be unpinned.

Required and executed configurations match in lexicographic diagnostic order:

1. semantic identity;
2. canonicalization identity;
3. digest; and
4. implementation identity, if the required commitment pins one.

Formally,

$$\text{match}(r, e) = \begin{cases} \text{SemanticMismatch}, & r_s \neq e_s, \\ \text{CanonicalizationMismatch}, & r_c \neq e_c, \\ \text{DigestMismatch}, & r_d \neq e_d, \\ \text{ImplementationMismatch}, & r_i = \text{some}(x) \wedge e_i \neq \text{some}(x), \\ \text{Matches}, & \text{otherwise.} \end{cases} \quad (37)$$

Reflexivity and functional determinism are checked. More importantly, `gateConfiguration` turns any mismatch into denial before returning an eligible result.

8.2 Three-way eligibility

Product evaluation returns exactly one of

$$\text{Eligible}(O), \quad \text{Denied}(\text{code}, \text{stage}), \quad \text{Indeterminate}(\text{code}, \text{stage}). \quad (38)$$

Eligible output O commits to reservation intents and obligations, carries counts bounded by 32, and carries canonical bytes bounded by 65,536. Lean proves the three-way partition and that any result classified as eligible has complete output commitments.

Checked product-safety claims

A configuration mismatch is never eligible. Every eligible value contains an `OutputCommitments` record satisfying its construction bounds. These are pure-contract properties; the truth of a domain's evidence and the durability of its eventual reservation remain outside this module.

8.3 Closed evaluators and fixed-context tightening

Rather than invent a universal policy language, the Lean record `ClosedEvaluator` packages a domain's own policy type P , context type C , evaluator $E : P \rightarrow C \rightarrow \text{Eligibility}$, tightening relation $\leqslant_P$, and output-refinement relation $\leqslant_O$. Its central law is

$$p_c \leqslant_P p_p \wedge E(p_c, c) = \text{Eligible}(o_c) \implies \exists o_p. E(p_p, c) = \text{Eligible}(o_p) \wedge o_c \leqslant_O o_p. \quad (39)$$

The context c is fixed. Tightening therefore cannot exploit a different clock, evidence snapshot, configuration, or state value. The theorem `fixed_context_tightening` simply exposes the law stored in the record; each concrete domain must supply the proof when it instantiates the evaluator.

8.4 Unit-aware checked arithmetic

A `UnitQuantity` stores a byte-exact unit, a natural amount, and a proof that the amount is no greater than

$$U_{64} = 2^{64} - 1 = 18,446,744,073,709,551,615.$$

Addition rejects unequal units and overflow. Subtraction rejects unequal units and underflow. Division rejects zero. Successful results preserve the unit and the natural-number operation exactly.

$$\text{checkedAdd}(x, y) = \text{ok}(z) \Rightarrow z.\text{amount} = x.\text{amount} + y.\text{amount} \leq U_{64}, \quad (40)$$

$$\text{checkedSub}(x, y) = \text{ok}(z) \Rightarrow y.\text{amount} \leq x.\text{amount}, \quad (41)$$

$$\text{checkedDiv}(x, 0) = \text{ok}(z) \Rightarrow \perp. \quad (42)$$

The production-refinement module separately proves that Aeneas-translated Rust checked addition, subtraction, multiplication, and division agree with natural-number specifications at the `u64` boundary.

9 Effect lifecycle as a guarded transition system

Authorization is repeatable; an external effect may not be. The lifecycle model begins after pure authority and product eligibility and describes when a runtime may reserve capacity, obtain credentials, enter a provider call, settle, or reconcile uncertainty.

9.1 States, operations, and gates

The state set is

$$\mathcal{S} = \{\text{DecisionRecorded}, \text{Reserved}, \text{ExecutionIntentRecorded}, \text{Executing}, \\ \text{Committed}, \text{Released}, \text{OutcomeUnknown}, \\ \text{ReconciledCommitted}, \text{ReconciledReleased}\}.$$

The transition function is total:

$$\delta : \text{Option}(\mathcal{S}) \times \mathcal{O} \times \mathcal{G} \rightarrow \mathcal{C}, \quad (43)$$

where $\mathcal{O}$ is the operation code, $\mathcal{G}$ is a Boolean record of gates, and $\mathcal{C}$ is either an applied state, observation-only result, terminal response, illegal transition, or stable rejection code.

The gate record includes core authorization, policy eligibility, configuration equality, revocation, expiry, capacity, execution intent, credential authorization, attempt presence, provider entry, cancellation, definite effect, definite non-effect, and reconciliation freshness and match.

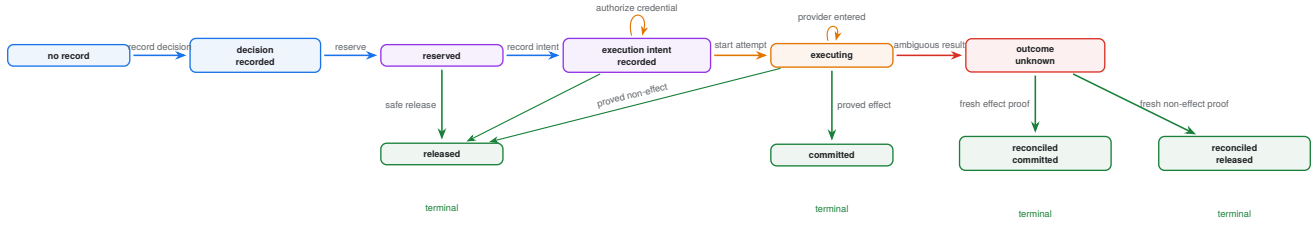


Figure 9: The effect lifecycle. Unknown outcome is not equivalent to release. It retains uncertainty until fresh reconciliation proves effect or non-effect. Self-loops represent ordered side-condition updates that do not change the lifecycle state value.

9.2 Safety ordering

The transition function encodes several non-commuting requirements.

Decision before reservation. The first state is created only if core authorization, product eligibility, configuration match, non-revocation, and non-expiry all hold.

Reservation before intent. Capacity is claimed before the execution intent is admitted.

Intent before credentials. Credential authorization is only defined at `ExecutionIntentRecorded` and requires the same configuration, revocation, expiry, and intent gates.

Credentials before attempts. Starting an attempt requires `credentialAuthorized=true`.

Attempt before provider entry. Marking provider entry requires an existing attempt and refuses duplicate entry.

Provider entry and effect proof before commit. Commit requires an attempt, provider-call entry, and definite effect.

Definite non-effect before risky release. Once executing, release requires both an attempt and definite non-effect.

Reconciliation after ambiguity. `OutcomeUnknown` cannot release directly. Only effect or non-effect reconciliation can reach a terminal state, and both require fresh matching reconciliation evidence.

Checked lifecycle ordering claims

Lean proves that starting an attempt implies credential authorization; provider-call entry implies an attempt; commit implies attempt, provider entry, and definite effect; unknown outcome cannot release; and only effect or non-effect reconciliation can terminate an unknown outcome. Configuration mismatch blocks both reservation and credential authorization. Terminal states never transition.

9.3 Capacity algebra

An additive ledger is $L = (c, m, a)$: ceiling, committed capacity, and active reservation. Its invariant is

$$\text{Valid}(L) \iff 0 < c \wedge m + a \leq c. \quad (44)$$

A request r is available only if

$$c \neq 0 \wedge r \neq 0 \wedge m + a \leq U_{64} \wedge m + a + r \leq U_{64} \wedge m + a + r \leq c. \quad (45)$$

The ledger operations are

$$\begin{aligned} \text{reserve}_r(c, m, a) &= (c, m, a + r), \\ \text{commit}_x(c, m, a) &= (c, m + x, a - x) \quad \text{when } x \leq a, \\ \text{release}_x(c, m, a) &= (c, m, a - x) \quad \text{when } x \leq a. \end{aligned} \quad (46)$$

Lean proves that every successful operation preserves Equation (44). Successful availability also implies positive ceiling and request, capacity containment, and absence of `u64` overflow.

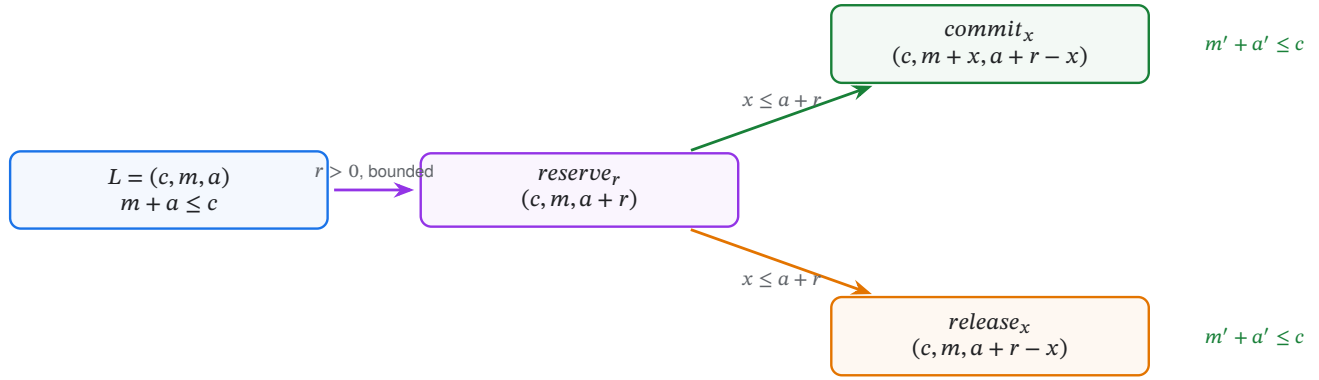


Figure 10: Capacity preservation. Reserve increases active capacity, commit moves active capacity into committed capacity, and release returns active capacity. Every successful step preserves the ceiling invariant.

Exclusive capacity uses a separate predicate:

$$\text{exclusiveAvailable}(\text{liveOwner}, \text{exactReplay}) = \neg \text{liveOwner} \vee \text{exactReplay}. \quad (47)$$

Thus the current owner may revisit the same exact operation, but a distinct claim cannot coexist with a live owner.

9.4 Replay classification

Replay is classified from record existence and commitment equality:

$$\text{replay}(\text{exists}, \text{equal}) = \begin{cases} \text{Absent}, & \neg \text{exists}, \\ \text{ExactReplay}, & \text{exists} \wedge \text{equal}, \\ \text{Conflict}, & \text{exists} \wedge \neg \text{equal}. \end{cases} \quad (48)$$

Lean checks that exact replay is stable, conflicting replay is never exact, and an absent record never claims an existing effect regardless of the irrelevant equality Boolean.

10 Refinement from shipping Rust to mathematical semantics

A proof about a handwritten model does not by itself prove a separately written implementation. Auths-Proof addresses this correspondence problem by isolating pure safe Rust functions, lowering them with Charon, translating them with Aeneas, and proving the generated Lean functions equivalent to the rich model. This resembles translation-validation thinking (Pnueli, Siegel, and Singerman 1998), but the target theorem relates translated program semantics to a domain specification rather than validating one compiler run.

10.1 Three semantic levels

The proof chain has three levels:

$$\text{validated Rust representation} \xrightarrow{\alpha} \text{rich semantic value} \xrightarrow{\llbracket - \rrbracket} \text{authorization meaning}. \quad (49)$$

The translated Rust function runs on the left. The readable specification runs on the middle value. Refinement proves that decisions agree. The denotational theorems then justify the security meaning of the rich relation.

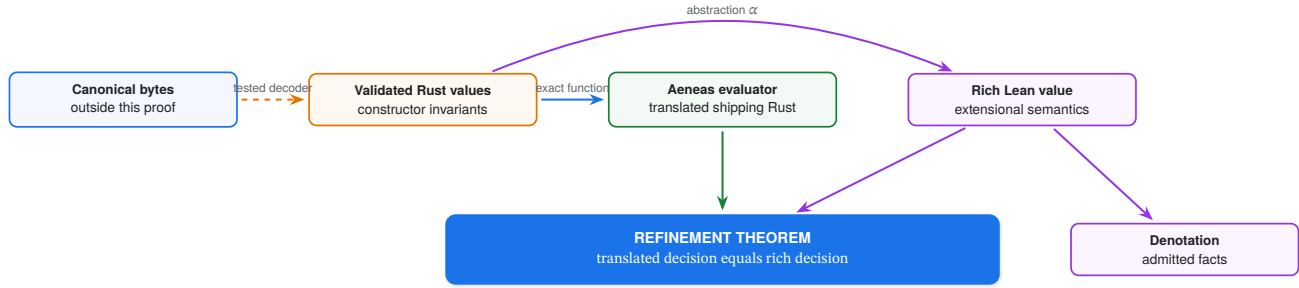


Figure 11: Refinement layers. Decoding is a separate boundary. Validated Rust values feed both the translated evaluator and an abstraction map into the rich model; the theorem equates their decisions.

10.2 Aeneas weakest-precondition judgments

The production bridge uses Aeneas’ functional translation and weakest- precondition notation. Schematically,

$$f(x) \{ y \mid Q(y) \} \quad (50)$$

states that running translated function f on x returns a result satisfying Q . Loop-bearing Rust functions are proved with decreasing natural measures and prefix invariants. For example, a membership loop over a vector maintains that every earlier index differs from the target; a subset loop maintains that every earlier child element has a corresponding parent element.

This is classic program logic applied to extracted safe Rust. The proof does not postulate that helper predicates behave semantically. It proves leaf specifications for byte equality, string bytes, profile equality, permission and audience membership and subset, digest membership, interval containment, constraints, budgets, status, and critical extensions, then composes them through the evaluator.

10.3 Representation maps

The production vocabulary instantiates opaque carriers as exact byte-oriented keys:

Rich carrier	Production key
Principal	UTF-8 byte list
Profile	(version, UTF-8 id bytes)
Permission	(capability bytes, resource bytes)
Audience	UTF-8 byte list
Digest	fixed byte list
Budget algebra	UTF-8 byte list
Status method	UTF-8 byte list
Assurance	UTF-8 byte list
Grant id	digest bytes
Critical extension	(id bytes, payload bytes)

Strings carry a premise that their UTF-8 byte size fits Aeneas’ u32 carrier. Windows carry a well-formedness proof. Freshness limits are positive. Selected profiles carry membership evidence. Sets carry boundedness and, where the Rust constructor guarantees it, canonicity.

Permissions and audiences map lists to extensional finite sets. The refinement therefore proves loop membership and subset against mapped keys. Critical extensions deliberately map to an ordered rich list, not a finite set, so positional equality remains exact. This is a good example of choosing the abstraction by the decision being refined rather than by superficial data shape.

10.4 Authority refinement theorems

There are three principal production-evaluator theorems.

Author scope. For validated parent and child `ScopeAuthorityView` values, the translated Rust pre-signing evaluator returns exactly `richAuthorScopeDecision`, including the first failing dimension.

Terminal coverage. For validated authority and action views, an anchoring premise, and a current optional-budget translation case, the translated Rust coverage evaluator returns the mapped rich `evaluateCoverage` decision.

Delegation. For validated parent and grant views plus anchoring, the translated Rust grant evaluator returns the mapped rich `evaluateGrant` decision. An accepted production transition is the exact field projection of the rich accepted next state.

In symbolic form, each has the shape

$$\text{Valid}(x) \wedge H(x) \implies \text{Aeneas}(f_{\text{Rust}})(x) = \gamma(f_{\text{Rich}}(\alpha(x))), \quad (51)$$

where H records explicit bridge premises, α is the representation map, and γ maps rich decisions back to production decision codes.

10.5 Lifecycle and bounded-policy refinement

The same pattern applies outside authority.

- Every rich lifecycle state, operation, and gate maps to its translated Rust counterpart. `translated_transition_refines_rich` proves equality for all combinations by exhaustive constructor analysis.
- Translated terminality, exclusive capacity, additive capacity, and replay classification agree with the rich functions.
- Translated configuration matching agrees with the pure product projection for all four input Booleans.
- Translated `u64` checked arithmetic refines natural-number arithmetic and correctly distinguishes successful results from overflow, underflow, or zero division.

These theorems turn fixed-width implementation concerns into explicit mathematical cases. Aeneas' `U64.checked_*` specifications provide the bitvector facts; Lean's arithmetic tactics discharge the natural-number consequences.

10.6 The explicit stale-translation theorem

The checked-in Aeneas translation of `optional_budget_covers` predates a shipping Rust correction. On one input class it returns the old, fail-open answer:

$$\text{translatedBudgetCovers}(\text{some}(b), \text{none}) = \text{true},$$

while the current rich and shipping semantics require

$$\neg \text{budgetCovers}(\text{some}(b), \text{none}). \quad (52)$$

The development does not hand-edit generated Lean or conceal the mismatch. It defines

$$\text{TranslatedBudgetCoverageCurrent}(B, R) \iff B = \text{none} \vee R \neq \text{none} \quad (53)$$

and requires this premise in the coverage refinement. Separately, `translated_budget_coverage_gap_is_the_absent_request` proves Equation (52) as an audited claim.

Executable disclosure. The coverage refinement intentionally excludes the present-ceiling/absent-request pair until the translation is regenerated. The mismatch is a theorem, not a comment. Once regenerated Lean matches the shipping Rust, the mismatch theorem should stop compiling and the exclusion premise should be removed atomically.

The authority bridge carries another explicit premise, `AuthorityStateAnchored`, because the checked-in authority translation predates a Rust root field. It states that a translated state either has a last grant or its supplied root equals its subject. This lets the rich state satisfy its local rootedness predicate without fabricating a translated field that is not present.

11 Assurance engineering around the proofs

Formal proof is only useful when the proposition, source closure, generated artifacts, and trusted assumptions remain identifiable. The `formal/` directory therefore contains an assurance system around Lean.

11.1 Public theorem inventory

`Auths.Theorems.theoremInventory` is an explicit list of the 121 public declarations discussed in Section 2.2. The compiled `AssuranceAudit` target looks up each exact declaration name in Lean’s environment. For each one it records

(name, kind, statement, transitive axioms).

The manifest additionally stores a digest of the reviewed theorem statement. Renaming, deleting, weakening, or changing a theorem changes the audit rather than silently satisfying a source-text search.

The global reviewed axiom allowlist is

{Classical.choice, Quot.sound, propext}. (54)

Individual theorems generally depend on subsets of this list. `sorryAx` is not allowed. The distinction between compiled and uncompiled artifacts matters: qualification templates contain placeholder axioms, but those templates are inventoried as uncompiled and are not imported into the audited theorem closure.

11.2 Qualified mechanical translation

The Aeneas qualification pins a complete tool and source configuration:

Component	Pinned value
Shipping Rust	1.97.1
Extraction Rust	nightly-2026-06-01
Lean	4.31.0
Charon	0.1.225 at commit 527ea8e...
Aeneas	commit 3a8586f...
Kani	0.67.0

The translated crates cover model predicates, the generated algebra kernel, the authority evaluators, bounded-policy primitives, and lifecycle primitives. The qualification requires zero opaque local functions and zero required compiled external axioms. External links are enumerated by exact Rust symbol. The primary standard-library semantic bridge is `String::as_bytes`, modeled as exact UTF-8 bytes within its carrier bound.

The pinned Aeneas runtime contains four general proof-support `sorry` declarations in `slice` and `string-iterator` modules. They are explicitly inventoried. The qualified `Auths` declarations do not transitively depend on them, and the compiled assurance audit would reject their appearance in a public claim’s axiom set.

11.3 Source closure

The qualification source-closure file hashes the Cargo manifests and lockfile, shipping Rust sources, algebra contract, formal qualification configuration, toolchain lock, and the repository code that drives the formal gate. Generated Lean files are separately listed. Clean qualification translates twice and requires byte-identical output before comparing with the committed artifacts.

This is not a proof that Charon or Aeneas is correct. It is a reproducibility and drift argument:

same pinned sources + same pinned tools $\implies$ same generated evidence, (55)

checked operationally rather than assumed informally.

11.4 Generated vectors and mutation witnesses

The Lean vector exporter produces three portable corpora.

All attenuation projections. Eleven Booleans yield $2^{11} = 2,048$ assignments. The expected answer is the generated conjunction.

Threshold states. For each required count $1 \leq k \leq 16$, the exporter enumerates every pair $0 \leq a \leq 16$ and $0 \leq u \leq 16 - a$, producing

$$16 \sum_{a=0}^{16} (17 - a) = 16 \cdot 153 = 2,448 \quad (56)$$

cases.

Rich semantic witnesses. The current exporter contains 26 focused vectors for interval boundaries, finite-set membership and inclusion, budget order and coverage, status freshness, and action-constraint constructors.

The mutation manifest contains 23 security-relevant operators, including reversing interval and subset directions, negating membership, accepting a different exact digest, ignoring algebra or method identity, treating an absent budget request as covered, accepting equal delegation depth, weakening principal or grant-id equality, and ignoring critical-extension payload changes.

The vectors are not substitutes for unbounded theorems. Their role is cross-language conformance and mutation sensitivity: a binding or optimized implementation must reproduce concrete consequences of the semantics.

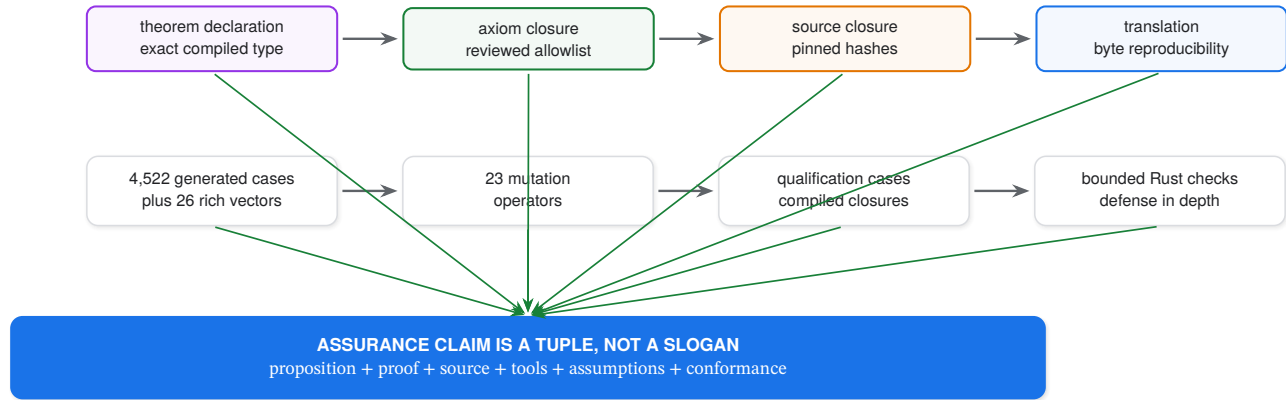


Figure 12: Assurance closure. A theorem is paired with its exact statement, transitive axioms, production source closure, translation inputs, and independent conformance evidence.

11.5 Why proof and model checking coexist

The Lean theorems are unbounded where their statements quantify over natural numbers, lists, scopes, or chain values. Bounded model checking remains useful for fixed-width implementation properties, unsafe states that should be unconstructible, panic freedom, and mutation killing. The two techniques answer different questions. Proof supplies universal consequences of a model; bounded checking explores the actual implementation within declared bounds (Kroening, Schrammel, and Tautschnig 2023).

12 Trusted computing base and residual assumptions

The formal result is strongest when its boundary is stated precisely. The trusted computing base and residual assumptions can be divided into five classes.

12.1 Logical foundation

The argument trusts the Lean kernel, the pinned Lean toolchain and libraries, and the reviewed foundational axioms in Equation (54). This is the usual small proof-checking base of an interactive theorem prover, not the whole IDE or tactic implementation.

12.2 Translation foundation

The argument trusts the pinned Rust, Charon, and Aeneas semantics and the reviewed external models. Aeneas is selected because it translates ownership-aware safe Rust into a functional representation suited to theorem proving (Ho and Protzenko 2022). Auths-Proof does not claim to verify Aeneas itself.

The Rust compiler remains trusted for correspondence between extracted Rust semantics and shipping machine code. This is narrower than a verified compiler chain such as CompCert (Leroy 2009) and different from language-level soundness work such as RustBelt (Jung et al. 2018).

12.3 Representation foundation

Aeneas exposes underlying carriers after erasing private Rust constructors. Refinement theorems therefore assume the invariants established by production constructors and validation:

- identifiers are canonical, nonempty, bounded byte sequences;
- counters and timestamps lie within fixed-width ranges;
- validity windows are well formed;
- collections are bounded and canonical where required;
- selected profiles are members of the allowed set;
- freshness limits are nonzero; and
- critical-extension identifiers are distinct and cardinality-bounded.

Many of these premises appear as Lean structures such as `AuthorityStateViewValid`, `GrantAuthorityViewValid`, `ActionAuthorityViewValid`, `ScopeAuthorityViewValid`, and `CriticalExtensionsCanonical`. The refinement theorem does not apply to an arbitrary malformed Lean value that safe Rust constructors could never create.

12.4 Environmental foundation

The formal core receives explicit evidence facts and gate Booleans. It assumes their interpretation is truthful. For example:

- `notRevoked=true` must come from a trustworthy revocation mechanism;
- `reconciliationFresh=true` must correspond to a domain-defined freshness policy;
- `definiteEffect=true` must be justified by provider-specific evidence;
- a configuration digest must be computed over the intended canonical bytes; and
- an assurance identifier must denote the policy the verifier believes it denotes.

The model proves what follows if these inputs are supplied. It does not prove the external world supplied them honestly.

12.5 Stateful implementation foundation

The lifecycle relation specifies legal transitions, but a concrete store must provide atomic compare-and-swap, durable ordering, isolation, and restart behavior. The capacity theorems apply to successful pure ledger steps. They do not by themselves prove that two concurrent transactions cannot both observe the same capacity or that a provider call and durable write are atomic.

Likewise, replay classification is pure. Exactly-once behavior requires the runtime to claim and record one logical effect at an enforcement boundary.

No whole-system shortcut. A proved transition relation does not make a database atomic. A proved coverage predicate does not make a signature valid. A proved refinement of pure Rust does not prove a network provider. The formal artifact is designed to make these composition boundaries explicit.

12.6 Claim matrix

Claim	Status in this artifact
Truth algebra and thresholds	Lean-proved; finite contract generated into Rust and Lean
Rich authority order and denotational safety	Lean-proved
Delegation root, depth, extension, and uniqueness laws	Lean-proved
Pure Rust authority evaluators	Aeneas-translated and refined under representation premises, with explicit current gap premises
Product commitment and arithmetic primitives	Lean-proved; selected Rust functions refined
Pure lifecycle kernel	Lean-proved and translated-Rust refined
Canonical decoding and signatures	Outside these theorems; separate validation boundary
Concrete stores and concurrency	Outside these theorems; conformance obligation
External evidence and provider outcomes	Explicit environmental assumption plus reconciliation semantics

13 Cross-disciplinary interpretation

The value of the formal development is easier to see when its mathematical, computer-science, and security readings are placed side by side.

13.1 Order theory: authorization as a monotone semantics

The map

$$\llbracket - \rrbracket : (\mathcal{S}, \leq) \rightarrow (\mathcal{P}(\mathcal{F}), \subseteq) \quad (57)$$

is monotone. Here $\mathcal{S}$ is the set of authority scopes and $\mathcal{F}$ the set of complete authorization facts. Structural attenuation is valuable because it is an efficiently decidable sufficient condition for semantic inclusion.

The product is not a homogeneous lattice. Some coordinates are ordinary orders, some are preorders, some use exact equality, and critical extensions have an initially unpinned top followed by exact lock-in. The system therefore uses a product of relations justified component by component rather than forcing every concern into one numeric lattice.

13.2 Denotational semantics: meaning before implementation

`Admits` gives scopes an extensional meaning independent of how Rust stores them. This makes representation refinement possible. A sorted vector, a finite set, and a canonical wire array may have different structures but can be related by the behaviors they authorize.

The distinction between `Exact(d)` and `Allowed({d})` illustrates the point: they are different syntax with equal denotation. Canonicalization chooses one representative when structural equality matters.

13.3 Type theory: invalid states as missing constructors

Several invariants are stored in types:

- inclusive windows contain a proof that start does not exceed finish;
- freshness limits contain a positivity proof;
- selected profiles contain membership evidence;
- unit quantities contain a u64 bound;
- output commitments contain count and byte bounds; and
- critical-extension sequences contain distinctness and length proofs.

This moves obligations from repeated runtime conditions into construction. The production bridge then explains which Rust constructors establish the corresponding premises.

13.4 Program logic: executable refinements

The weakest-precondition lemmas have the shape familiar from Hoare logic:

$$\{P\} f \{Q\}. \quad (58)$$

Loop invariants connect imperative traversal to extensional membership and subset. The large authority refinement proofs then compose these local contracts through the translated evaluator. This is how the artifact advances from “the model has good laws” to “the selected shipping predicate returns the model’s decision.”

13.5 Distributed systems: uncertainty is state

The truth value `Indeterminate` and lifecycle state `OutcomeUnknown` address different uncertainties.

- `Indeterminate` means authorization evidence is insufficient but could resolve.
- `OutcomeUnknown` means an execution attempt may or may not have produced an effect.

Neither is permission. The first cannot become an authorized command without new trustworthy evidence. The second cannot become released capacity without fresh reconciliation. Preserving these distinctions prevents fail-open logic at both decision and effect boundaries.

13.6 Security engineering: negative-space theorems

Many of the strongest results state that a dangerous shortcut is impossible:

- an unrooted authority authorizes nothing;
- a foreign issuer falsifies the root dimension;
- a changed critical extension falsifies the extension dimension;
- a narrowed scope cannot admit a parent-rejected fact;
- a start attempt cannot succeed without credential authorization;
- a commit cannot succeed without provider entry and effect proof;
- an unknown outcome cannot be released; and
- an absent replay record cannot claim an existing effect.

These are negative-space theorems: they rule out classes of bad executions rather than merely exhibiting a good example.

14 Limitations and open proof obligations

The artifact is unusually explicit about its incomplete edges. Several are important research and engineering directions.

14.1 Regenerate the stale authority translation

The optional-budget and root-field bridge premises should disappear after the checked-in Aeneas output is regenerated from current shipping Rust. The existing mismatch theorem makes this a fail-loud change: successful regeneration should invalidate the stale theorem rather than leave dead documentation behind.

14.2 Close constructor-to-view refinement

Representation-validity premises are reviewed and tested, but the strongest chain would prove that every value produced by public safe Rust constructors satisfies the corresponding Lean validity structure and that every lossless view preserves the fields consumed by the evaluator.

14.3 Strengthen plan semantics

The composition algebra has strong truth and threshold results, but the current `Plan` theorems about leaf visits and cost are definitional. A richer development could define an instrumented recursive evaluator and prove occurrence-sensitive visitation, cost bounds, duplicate policy, arbitrary permutation invariance, and diagnostic stability for complete plans.

14.4 Connect state-machine relations to stores

The lifecycle kernel proves legal pure transitions. Concrete exactly-once claims additionally need a refinement from transactional store operations and crash histories to the pure state relation, including concurrent final capacity, restart, and reconciliation.

14.5 Separate information-flow properties

Deterministic first-failure diagnostics do not establish noninterference, constant-time behavior, or resistance to policy probing. Those require a separate observation model and attacker relation.

14.6 Expand domain-specific evaluator proofs vertically

`ClosedEvaluator` defines the reusable proof obligation for policy tightening, but each domain must instantiate it with its own exact action, evidence, reservation, obligation, and output relations. Shared product mechanisms should follow proven semantic identity across completed domains rather than precede it.

15 Related work

Auths-Proof sits between authorization logic, proof-carrying systems, and systems verification.

Authorization logics distinguish authenticated statements from the authority to act (Abadi et al. 1993). Trust-management systems make credentials and policy composition explicit (Blaze, Feigenbaum, and Lacy 1996). Macaroons demonstrate decentralized attenuation through contextual caveats (Birgisson et al. 2014). Auths-Proof shares the monotone-restriction goal but formalizes a closed heterogeneous scope, exact terminal action coverage, and a separate effect lifecycle.

Proof-carrying code asks a producer to supply machine-checkable evidence that a consumer validates against a safety policy (Necula 1997). Proof-carrying authentication applies related ideas to authorization (Appel and Felten 1999). Auths-Proof is closer in spirit to proof-carrying authentication than to code safety, but its shipped proof object is not a

Lean proof term for every action. Lean establishes the kernel laws; runtime objects carry signed authorization evidence checked by that kernel.

Large verification projects such as CompCert and seL4 demonstrate the value of explicit refinement chains and trusted computing bases (Leroy 2009; Klein et al. 2009). Auths-Proof is narrower. It verifies security-critical pure semantics and selected production functions, not an operating system or compiler. Aeneas makes the production linkage practical for ownership-aware safe Rust (Ho and Protzenko 2022), while work such as RustBelt and Verus attacks complementary language-soundness and deductive-verification problems (Jung et al. 2018; Lattuada et al. 2024).

16 Conclusion

The formal work in Auths-Proof is best understood as a sequence of semantic compressions.

1. A complex authorization scope is compressed into a decidable product relation without reducing every dimension to a scalar.
2. That structural relation is justified by a denotational theorem: child admission is a subset of parent admission.
3. Delegation adds trust-root linkage, exact lineage, deterministic state construction, critical-extension lock-in, and a well-founded depth measure.
4. Three-valued composition preserves the distinction between denial, unresolved evidence, and authorization.
5. Product and lifecycle models carry the result toward irreversible effects through commitments, checked arithmetic, reservations, credentials, attempts, provider entry, unknown outcomes, and reconciliation.
6. Mechanical Rust translation and refinement connect the readable model to selected shipping predicates under explicit representation assumptions.
7. The assurance manifest binds each public English claim to an exact compiled proposition, axiom set, toolchain, source closure, and evidence set.

The central theorem remains Equation (1): authority can move downward without creating new admitted facts. Its practical power comes from the surrounding work. Root preservation prevents re-anchoring. Strict depth prevents infinite delegation. Pinned extensions prevent semantic stripping. Exact diagnostics make decisions stable. Lifecycle invariants prevent authorization from being mistaken for effect completion. Refinement prevents the mathematical model from floating free of production Rust.

The result is neither a decorative proof nor a claim that the entire system is verified. It is a disciplined formal boundary: small enough to audit, rich enough to express the dangerous cases, mechanically connected to executable semantics, and honest about what remains outside.

Appendix A. Complete scope relation

For reference, the complete scope order is

$$\begin{aligned}
 S_c \leq S_p &\iff \Phi_c \leq \Phi_p \\
 &\quad \wedge P_c \subseteq P_p \\
 &\quad \wedge I_c \subseteq I_p \\
 &\quad \wedge A_c \subseteq A_p \\
 &\quad \wedge C_c \leq C_p \\
 &\quad \wedge B_c \leq B_p \\
 &\quad \wedge T_c \leq T_p \\
 &\quad \wedge Q_c = Q_p \\
 &\quad \wedge X_c \leq X_p.
 \end{aligned} \tag{A.1}$$

Dimension	Child below parent when	Coverage consequence
Profile	same root set; first selection or same selection	child-covered profile is parent-covered
Permissions	child set is a subset	child member is parent member
Validity	child interval is contained	child-contained action window is parent-contained
Audiences	child set is a subset	child member is parent member
Action body	allowed digest denotation is included	child-allowed digest is parent-allowed
Budget	parent unbounded, or same algebra with lower child ceiling	request covered by child is covered by parent
Status	parent expiry-only, or same method with no looser child age	evidence satisfying child satisfies parent
Assurance	exact identity equality	evidence assurance transfers by equality
Extensions	parent unpinned, or exact pinned sequence equality	chain cannot strip or alter pinned constraints

Appendix B. Delegation proof skeleton

Let q_0 be a parent, h a grant, i its identifier, and q_1 a proposed child.

1. Prove $\text{rooted}(q_0)$ and $h.\text{issuer} = q_0.\text{subject}$.
2. Prove $h.\text{parent} = q_0.\text{lastGrant}$.
3. Prove $0 < q_0.\text{depth}$ and $h.\text{depth} < q_0.\text{depth}$.
4. Prove profile permission, window, audience, action, budget, status, assurance, and extension checks.
5. Construct $q^* = \text{acceptedNextState}(q_0, i, h)$.
6. Show $q_1 = q^*$.

Then

$$\text{delegates}(q_0, i, h, q_1)$$

and the checked consequences include

$$\begin{aligned}
 q_1.\text{root} &= q_0.\text{root}, & q_1.\text{subject} &= h.\text{subject}, & q_1.\text{lastGrant} &= \text{some}(i), \\
 q_1.\text{depth} &< q_0.\text{depth}, & q_1.\text{scope} &\leq q_0.\text{scope}, & \text{rooted}(q_1).
 \end{aligned} \tag{B.1}$$

Appendix C. Lifecycle transition summary

Current state	Operation	Essential success gates	Result
none	record decision	authorized, eligible, configuration match, live	decision recorded
decision recorded	reserve	configuration match, live, capacity	reserved
reserved	record intent	configuration match, live, intent present	intent recorded
intent recorded	authorize credential	configuration match, live, intent, not already authorized	intent recorded
intent recorded	start attempt	configuration match, live, intent, credential authorized	executing
executing	mark provider entered	attempt present, not already entered	executing
executing	commit	attempt, provider entered, definite effect	committed
reserved	release	no attempt; cancellation or definite non-effect	released
intent recorded	release	no attempt	released
executing	release	attempt and definite non-effect	released
executing	mark unknown	attempt present	outcome unknown
outcome unknown	reconcile effect	fresh matching reconciliation	reconciled committed
outcome unknown	reconcile non-effect	fresh matching reconciliation	reconciled released
outcome unknown	reconcile inconclusive	fresh matching reconciliation	observation only

All terminal states return `terminal` for every later operation. All omitted state-operation pairs return `illegalTransition` or a more specific failed gate code according to the definition.

Appendix D. Public theorem families

The table below maps mathematical claims to representative Lean declarations. It is a guide, not a replacement for `formal/Auths/Theorems.lean` or the assurance manifest.

Mathematical claim	Representative declaration
Component orders are reflexive/transitive	<code>finiteSet_subset_*, window_contained_*, action_constraint_*, budget_*, status_*, profile_*, extensions_*</code>
Canonical scopes are antisymmetric	<code>scope_le_canonical_antisymmetry</code>
Structural attenuation is semantically sound	<code>structural_scope_le_implies_semantic_attenuation</code>
Complete admission is downward closed	<code>complete_admission_downward_closed</code>
Accepted edges never widen	<code>delegate_never_widens</code>
Accepted chains are finite	<code>remaining_depth_well_founded, finite_delegation_chain</code>
Root identity persists	<code>delegate_preserves_root, chain_preserves_root</code>
Pinned extensions persist	<code>delegate_preserves_pinned_extensions, chain_preserves_pinned_extensions</code>

Mathematical claim	Representative declaration
Grant evaluation equals delegation relation	<code>apply_grant_success_iff_delegates</code>
Accepted child is unique	<code>apply_grant_success_unique</code>
Coverage decision equals terminal coverage	<code>coverage_decision_ok_iff_covers</code>
Shipping author evaluator matches rich semantics	<code>translated_rust_refines_rich_spec</code>
Shipping coverage evaluator matches rich semantics	<code>translated_coverage_refines_rich_spec</code>
Shipping delegation evaluator matches rich semantics	<code>translated_delegation_refines_rich_spec</code>
Configuration mismatch denies	<code>product_contract_configuration_safety</code>
Successful checked arithmetic is safe	<code>checked_add_never_wraps, checked_sub_never_underflows,</code> <code>checked_div_rejects_zero</code>
Capacity steps preserve validity	<code>reserve_preserves_capacity, commit_preserves_capacity,</code> <code>release_preserves_capacity</code>
Effect ordering is enforced	<code>start_attempt_requires_credential,</code> <code>provider_call_requires_attempt,</code> <code>commit_requires_provider_entry_and_effect</code>
Unknown outcome requires reconciliation	<code>outcome_unknown_cannot_release,</code> <code>outcome_unknown_only_reconciliation_can_terminate</code>
Translated lifecycle equals rich lifecycle	<code>translated_transition_refines_rich</code>
Threshold result matches count region	<code>authorized_implies_threshold_met,</code> <code>denied_implies_threshold_impossible,</code> <code>indeterminate_implies_threshold_reachable</code>

Appendix E. Module dependency guide

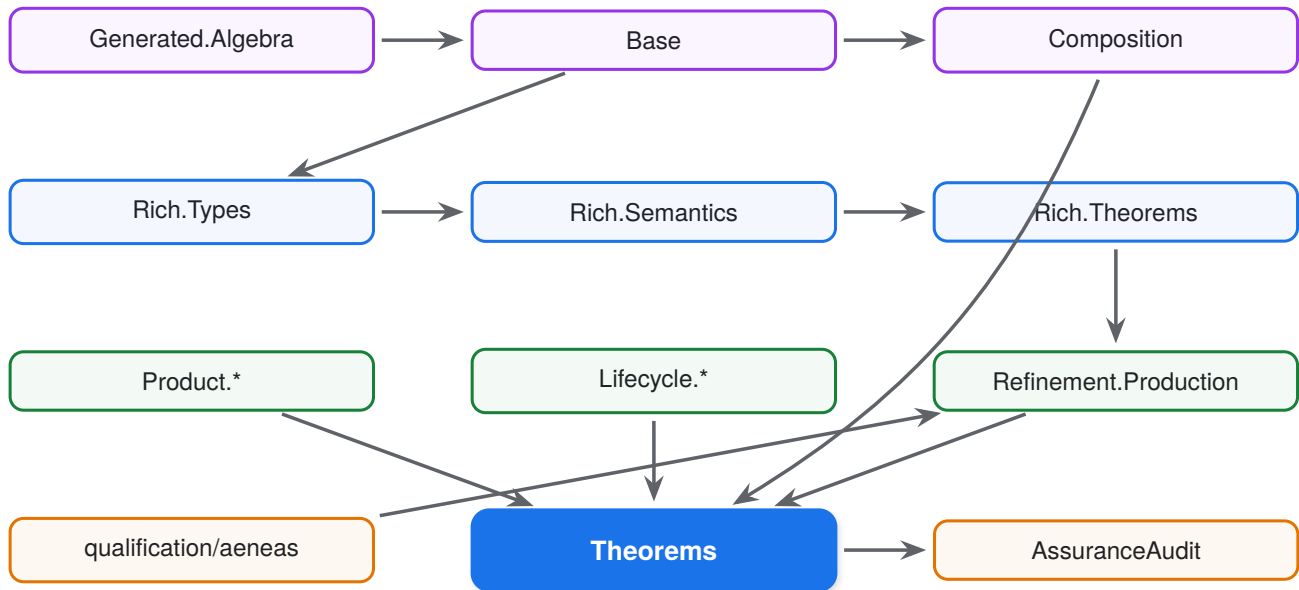


Figure 13: A reader’s dependency guide. Arrows indicate conceptual import flow toward the public theorem inventory and compiled assurance audit.

References

- Abadi, Martín, Michael Burrows, Butler Lampson, and Gordon Plotkin. 1993. “A Calculus for Access Control in Distributed Systems.” *ACM Transactions on Programming Languages and Systems* 15 (4): 706–34. <https://doi.org/10.1145/155183.155225>.
- Appel, Andrew W., and Edward W. Felten. 1999. “Proof-Carrying Authentication.” In *Proceedings of the 6th ACM Conference on Computer and Communications Security*, 52–62. <https://doi.org/10.1145/319709.319718>.

- Birgisson, Arnar, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentczner. 2014. “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud.” In *Network and Distributed System Security Symposium*. Internet Society.
- Blaze, Matt, Joan Feigenbaum, and Jack Lacy. 1996. “Decentralized Trust Management.” In *Proceedings of the 1996 IEEE Symposium on Security and Privacy*, 164–73. <https://doi.org/10.1109/SECPRI.1996.502679>.
- Davey, B. A., and H. A. Priestley. 2002. *Introduction to Lattices and Order*. Second. Cambridge University Press. <https://doi.org/10.1017/CBO9780511809088>.
- Dijkstra, Edsger W. 1975. “Guarded Commands, Nondeterminacy and Formal Derivation of Programs.” *Communications of the ACM* 18 (8): 453–57. <https://doi.org/10.1145/360933.360975>.
- Ho, Son, and Jonathan Protzenko. 2022. “Aeneas: Rust Verification by Functional Translation.” *Proceedings of the ACM on Programming Languages* 6 (ICFP): 711–41. <https://doi.org/10.1145/3547647>.
- Hoare, C. A. R. 1969. “An Axiomatic Basis for Computer Programming.” *Communications of the ACM* 12 (10): 576–80. <https://doi.org/10.1145/363235.363259>.
- Jung, Ralf, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2018. “RustBelt: Securing the Foundations of the Rust Programming Language.” *Proceedings of the ACM on Programming Languages* 2 (POPL): 66:1–34. <https://doi.org/10.1145/3158154>.
- Klein, Gerwin, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, et al. 2009. “seL4: Formal Verification of an OS Kernel.” In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles*, 207–20. <https://doi.org/10.1145/1629575.1629596>.
- Kroening, Daniel, Peter Schrammel, and Michael Tautschnig. 2023. “CBMC: The C Bounded Model Checker.” *arXiv Preprint arXiv:2302.02384*. <https://doi.org/10.48550/arXiv.2302.02384>.
- Lattuada, Andrea, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, et al. 2024. “Verus: A Practical Foundation for Systems Verification.” In *Proceedings of the 30th ACM Symposium on Operating Systems Principles*, 438–54. <https://doi.org/10.1145/3694715.3695952>.
- Leroy, Xavier. 2009. “Formal Verification of a Realistic Compiler.” *Communications of the ACM* 52 (7): 107–15. <https://doi.org/10.1145/1538788.1538814>.
- Moura, Leonardo de, and Sebastian Ullrich. 2021. “The Lean 4 Theorem Prover and Programming Language.” In *Automated Deduction – CADE 28*, 12699:625–35. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-030-79876-5_37.
- Necula, George C. 1997. “Proof-Carrying Code.” In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 106–19. <https://doi.org/10.1145/263699.263712>.
- Pnueli, Amir, Michael Siegel, and Eli Singerman. 1998. “Translation Validation.” In *Tools and Algorithms for the Construction and Analysis of Systems*, 1384:151–66. Lecture Notes in Computer Science. Springer. <https://doi.org/10.1007/BFb0054170>.